Supplementary Material for "Evaluating and Training IEC 61131-3 Structured Text Generators with Compiler, Invariant, and Scenario Feedback"

Anonymous Author(s)

TABLE S-I
COMPILE-AND-INVARIANT PASS@k (OVERALL), JUNE 2026 REPLICATION WINDOW. n=10, TEMPERATURE 0.8. SAMPLING HELPS GROK-4.3† (MORE RECOVERABLE VARIATION) MORE THAN CLAUDE (MORE ALL-OR-NONE). †SERVED BY `grok-4.3` (`reasoning_effort=none`) VIA THE RETIRED `grok-3` SLUG (SECTION V OF THE MAIN PAPER).

Model	pass@1	pass@3	pass@5	pass@10
grok-4.3†	0.645	0.817	0.881	0.955
claude-sonnet-4-6	0.564	0.611	0.635	0.682
gemini-2.5-flash	0.095	0.172	0.204	0.273
gpt-4o	0.091	0.161	0.196	0.227

This document contains the supplementary tables, figures, and appendices for the main paper. Section, table, figure, and equation numbers written in Roman/Arabic numerals without an "S" prefix (e.g. Table II, Section V-F, Eq. (2)) refer to the main paper.

S.I. API PANEL: PASS@k DETAIL AND TIER BREAKDOWN

This section gives the per-k and per-tier detail for the June 2026 replication window (Table S-XVIII; n=10 samples per task, temperature 0.8). Table S-I reports compile-and-invariant pass@k for $k \in \{1, 3, 5, 10\}$, Fig. S-1 breaks pass@1 down by structural-complexity tier, and Fig. S-2 plots the pass@k curves. The tier walls shown here (gemini no medium-tier verified task, gpt-4o no hard-tier task) are specific to this June window; the retained July panel records a single marginal success in each (Section V of the main paper).

S.II. FAILURE-CATEGORY COMPOSITION (JULY PANEL)

Table S-II reports the failure-category composition of the retained July n=10 panel (220 samples per model over the 22 tasks), categorizing each sample as *verified*, *unsafe* (MATIEC-accepted and translator-analyzable yet violating at least one supplied invariant), *compile_error* (rejected by MATIEC), *parse_error* (rejected by the ST→SMV parser), or *translate_error* (parses but is not lowerable to SMV); the last two are the finer split the headline collapses into *front-end*. Two observations hold. First, weak-model failures are dominated by *validity*, not by unsafe logic: gemini fails 176 of 220 at parse and gpt-4o 189 of 220 at parse or compile, consistent with the dual-oracle-overestimation result of RQ2. Second, *unsafe* programs—MATIEC-accepted and analyzable yet violating a supplied invariant—are rare but present across the panel (gpt-4o 7, grok-4.3† 2, claude 1, gemini 0; concentrated in `M02`/`M05`/`M06`/`E04`), precisely the failure mode the safety oracle exists to catch and which a compile-only benchmark would score as a pass. The *verified* column equals the compile-and-invariant pass@1 of Section V of the main paper (each task contributes 10 samples). The June-window taxonomy is in the artifact (`docs/07`).

Author identities withheld for double-blind review.

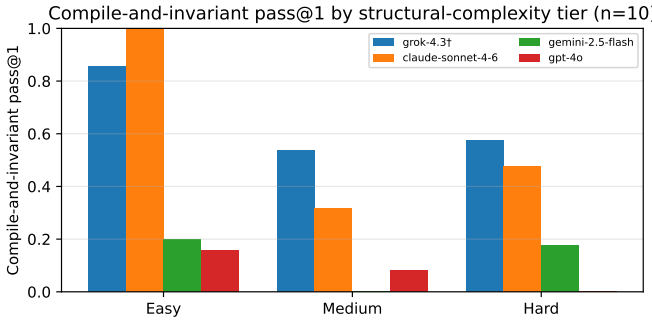


Fig. S-1. Compile-and-invariant pass@1 by author-assigned structural-complexity tier (n=10, 22 tasks), June 2026 window: gemini-2.5-flash verifies no medium-tier task and gpt-4o no hard-tier task; note the hard tier has only four tasks. In the retained July panel each clears a single such task (Section V of the main paper).

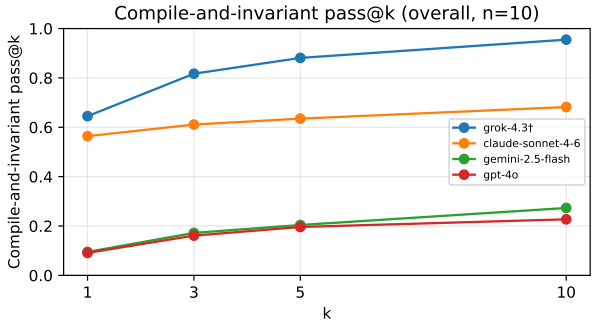


Fig. S-2. Compile-and-invariant pass@k (overall, n=10), June 2026 window. grok-4.3† gains strongly with k (misses often partial and resampling-recoverable); claude-sonnet-4-6 is nearly flat (misses tend to be all-or-none). Consistent with—but not established as—stochastic vs. systematic failure; may reflect provider-side decoding (Section V of the main paper).

S.III. PER-TASK VERDICT MATRIX

Table S-III gives the per-task *verified* / *task-valid* sample counts (out of n=10) for each task and model on the retained

TABLE S-II
FAILURE-CATEGORY COMPOSITION OF THE RETAINED JULY n=10 PANEL (220 SAMPLES PER MODEL OVER THE 22 TASKS; NOT THE TRAINED POLICIES). PARSE AND TRANSL. ARE THE FINER VALIDITY SPLIT (PARSE_ERROR, TRANSLATE_ERROR) THE HEADLINE AGGREGATES INTO FRONT-END; UNSAFE = MATIEC-ACCEPTED AND ANALYZABLE BUT VIOLATING ≥ 1 SUPPLIED INVARIANT. ROWS SUM TO 220; THE VERIFIED RATE EQUALS COMPILE-AND-INVARIANT PASS@1. †GROK-4.3 VIA THE RETIRED GROK-3 SLUG.

Model	Verified	Unsafe	Compile	Parse	Transl.	rate
grok-4.3†	149	2	17	40	12	0.677
claude-sonnet-4-6	122	1	1	74	22	0.555
gemini-2.5-flash	22	0	16	176	6	0.100
gpt-4o	24	7	54	135	0	0.109

TABLE S-III
PER-TASK JULY SAMPLE COUNTS (OUT OF n=10), REPORTED AS VERIFIED / TASK-VALID (COMPILE-AND-INVARIANT / ALL-CHECKS-PASS). TASKS IN TIER ORDER; SEE TABLE S-XI FOR FULL NAMES. †GROK-4.3 VIA THE RETIRED GROK-3 SLUG.

Task	Tier	grok-4.3†	claude	gemini	gpt-4o
E01	easy	10/10	10/10	2/2	1/1
E02	easy	7/7	9/9	0/0	0/0
E03	easy	10/10	10/10	0/0	0/0
E04	easy	8/8	10/10	0/0	1/1
E05	easy	9/9	10/10	5/5	1/1
E06	easy	8/8	10/10	5/5	6/6
E07	easy	10/10	10/10	5/5	2/2
M01	med.	10/10	0/0	0/0	0/0
M02	med.	4/3	3/3	0/0	0/0
M03	med.	1/1	0/0	0/0	1/0
M04	med.	10/4	0/0	0/0	0/0
M05	med.	4/3	1/0	0/0	0/0
M06	med.	4/3	0/0	0/0	0/0
M07	med.	6/6	0/0	0/0	0/0
M08	med.	9/9	10/10	0/0	0/0
M09	med.	10/6	10/0	0/0	0/0
M10	med.	0/0	0/0	0/0	3/1
M11	med.	4/4	10/10	2/2	8/8
H01	hard	5/5	0/0	0/0	0/0
H02	hard	10/4	10/0	0/0	0/0
H03	hard	0/0	0/0	0/0	0/0
H04	hard	10/10	9/9	3/3	1/1
Total		**149/130**	**122/101**	**22/22**	**24/21**

July panel, the raw data behind the aggregate pass@k of Section V of the main paper. Verified column sums are $149/122/22/24$ over 220 task-samples, matching the reported compile-and-invariant pass@1 of $0.677/0.555/0.100/0.109$; task-valid sums are $130/101/22/21$. The matrix substantiates the stochastic-vs-systematic reading in RQ1: grok-4.3†'s misses are often *partial* (e.g. M02=4, M05=4, M06=4, H01=5), which resampling recovers, whereas claude's are typically all-or-nothing (e.g. M08=10 vs. M01=0). It also shows that gemini and gpt-4o verify almost nothing beyond the easy tier, and that the scenario check bites even when invariants pass (grok H02 10/4, claude M09 10/0). Per-task counts for the June window are in the artifact (docs/07).

S.IV. OUTPUT-ACTIVITY PROBE OF THE TRAINED POLICIES

Section V-D of the main paper summarizes the output-activity probe; Table S-IV reports it in full.

TABLE S-IV
OUTPUT-ACTIVITY PROBE OF TRAINED POLICIES (n=10 COMPLETIONS PER TASK, THREE TRAINING SEEDS) ON THE TASKS' SCENARIO INPUT SEQUENCES. "ACT. (PARSED)" = FRACTION OF PARSEABLE COMPLETIONS THAT EVER DRIVE AN OUTPUT (DENOMINATOR VARIES WITH THE PARSE RATE); "ACT. (ALL)" COUNTS NONPARSEABLE COMPLETIONS AS INACTIVE, SO DENOMINATORS ARE COMPARABLE ACROSS CONDITIONS. ALL-OFF≡ = FRACTION OF PARSED COMPLETIONS BEHAVIORALLY EQUIVALENT TO THE INERT ALL-OFF POLICY. EVER-DRIVING-AN-OUTPUT IS A WEAK CRITERION (AN IRRELEVANT ACTIVATION COUNTS); THE SCENARIO-PASS COLUMN IS THE STRICTER FUNCTIONAL MEASURE. BOLD MARKS THE BASE→SFT→SFT-V2 PROGRESSION ENDPOINT DISCUSSED IN THE TEXT AND THE UNCONDITIONAL-ACTIVITY MAXIMUM.

condition	parse	scen. pass	act. (parsed)	act. (all)	all-off≡
base	0.458	0.202	0.773	0.354	0.227
SFT (3-epoch)	0.468	0.244	0.944	0.442	0.056
SFT→RL	0.503	0.245	0.943	0.474	0.058
SFT-v2	0.473	0.373	**0.975**	0.461	**0.025**
SFT-lite	0.579	0.267	0.861	0.498	0.134
RL-lite (inv. reward)	0.671	0.308	0.841	0.565	0.159
RL (func. reward)	0.653	0.318	0.887	**0.580**	0.113
SFT-hardgen	0.503	0.308	0.987	0.497	0.013

TABLE S-V
CONSOLIDATED EXPERIMENTAL DESIGN: THE UNTRAINED BASE AND THE NINE TRAINED 7B CONDITIONS (BASE MODEL QWEN2.5-CODER-7B-INSTRUCT WITH 4-BIT QLORA; HYPERPARAMETERS IN SEC. S.IX). SFT CORPORA: 158 = CANONICAL PROCEDURAL INSTANCES; 208 = 158 + 50 HARDER-FAMILY INSTANCES (GENERATION-ONLY); 369 = 208 + 161 COUNTEREXAMPLE-REPAIR PAIRS. FOR RL ROWS THE CORPUS/EPOCHS COLUMNS DESCRIBE THE SFT WARM START. ALL RL IS GRPO ON THE 42–47 HELD-OUT FRONTIER PROMPTS, RE-SELECTED PER WARM-START POLICY (SECTION VI-C OF THE MAIN PAPER), WITH UP TO 30 UPDATE STEPS FROM AN SFT WARM START AND 50 FROM BASE. EVERY CONDITION IS EVALUATED AT n=10 PER TASK, TEMPERATURE 0.8, ON THE 22-TASK BENCHMARK UNDER BOTH METRICS; THE 1.5B/14B RUNS OF TABLE S-VIII REUSE THE 3-EPOCH RECIPE.

condition	SFT corpus	epochs	RL reward	seeds	results
base	—	—	—	5	Tab. S-VI; Tab. S-XIV
SFT (3-epoch)	158	3	—	3	Tab. S-VI; Tab. S-XIV
SFT-lite	158	1	—	5	Tab. S-VII; Tab. S-XIV
SFT-v2	369	3	—	3	Tab. S-VI; Tab. S-XIV
SFT-hardgen (gen.-only + hard)	208	3	—	3	Tab. S-X; Tab. S-IV
SFT→RL	158	3	Eq. (2) of the main paper	3	Tab. S-VI; Tab. S-XIV
RL-lite	158	1	Eq. (2) of the main paper	5	Tab. S-VII; Tab. S-XIV
RL (func. reward)	158	1	Eq. (3) of the main paper	3	Tab. S-XIV
RL (crit. gated)	158	1	Eq. (3) hard-gated on critical properties (Sec. S.XVIII)	3	Tab. S-XIV
RL-from-base	—	—	Eq. (2) of the main paper	3	Tab. S-VI

S.V. EXPERIMENTAL DESIGN AND COMPILE-AND-INVARIANT TRAINING DETAIL

Table S-V consolidates the experimental design of the training conditions compared in Section VII of the main paper. Table S-VI is the canonical 7B compile-and-invariant table (its verified columns also appear in Table III of the main paper), and Table S-VII isolates the entropy-preserving warm-start comparison discussed in Section VII-B of the main paper.

S.VI. MODEL-SIZE SCALING

Table S-VIII and Fig. S-3 report the 1.5B/7B/14B scaling study discussed in Section VII-C of the main paper.

S.VII. REPAIR FEEDBACK-CHANNEL CONTROLS

Table S-IX reports the repair feedback-channel controls discussed in Section VII-D of the main paper.

Full repair-control protocol. We ran the trace-ablation controls (three seeds, evaluated on a *held-out* bug type—models trained on {weaken-OR-guard, loosen-comparison-bound} bugs and tested on a held-out forces-output-true bug

TABLE S-VI
VERIFIER-GROUNDED TRAINING ON QWEN2.5-CODER-7B, COMPILE-AND-INVARIANT PASS@k (ROUND-3 RE-SCORING, THE RELEASED-PROGRAMS RUN). MEAN±STD OVER INDEPENDENT SEEDED RUNS: FIVE SEEDS FOR THE BASE ROW, THREE FOR ALL OTHER ROWS; n=10 COMPLETIONS PER TASK WITHIN EACH SEED. CONDITIONS: BASE; 3-EPOCH SFT; SFT→RL; STRENGTHENED-DATA SFT (V2); AND RL FROM THE BASE MODEL. INTERPRETATION IS IN THE BODY; THE ENTROPY-PRESERVING RL-LITE RESULT (0.389) IS IN TABLE S-VII.

Condition	pass@1	pass@10
base	0.079±0.020	0.373±0.018
SFT (procedural-domain)	0.311±0.025	0.455±0.037
SFT→RL (dual-oracle)	0.352±0.009	0.470±0.021
SFT-v2 (strengthened data)	**0.395±0.039**	**0.470±0.057**
RL-from-base (frontier prompts)	0.070±0.015	0.333±0.021

TABLE S-VII
RL GAIN AFTER THE 1-EPOCH ENTROPY-PRESERVING WARM-START (7B). COMPILE-AND-INVARIANT PASS@k, MEAN±STD OVER FIVE TRAINING SEEDS; n=10 COMPLETIONS PER TASK WITHIN EACH SEED. THE TWO ROWS DIFFER IN EPOCHS, LEARNING RATE AND ENTROPY, SO THIS IS AN ASSOCIATION, NOT AN ENTROPY-CONTROLLED COMPARISON.

Condition	pass@1	pass@10
SFT-lite (1 epoch, entropy-preserving)	0.242±0.040	0.518±0.022
RL-lite (frontier GRPO)	**0.389±0.021**	**0.609±0.022**

on 17 of the 22 tasks, task-valid pass@1; Table S-IX). Only 17 tasks are scored because the held-out mutation forces an output `TRUE` by mutating an existing `:= FALSE` assignment in the reference, and five references contain no such assignment. This repair benchmark is also a different—and easier—conditional task than generation: the model is *given* the buggy, near-correct program to fix, which is why the untrained base scores 0.388 here versus its 0.045 task-valid pass@1 on generation. Training on repair-format data helps (every variant > generic SFT 0.461 > untrained base 0.388), but the *content* of the feedback does not: the full correct counterexample (0.514) does not exceed no counterexample at all (0.553)—three seeds do not resolve this difference and we make no equivalence claim—and a *mismatched* trace shuffled from another task (0.680) is if anything higher—the opposite of what a trace-driven mechanism predicts. These results suggest that the repair-format data (buggy code paired with its correction), rather than the correct trace content, may be responsible; because a shuffled trace also changes sequence length, token distribution, and formatting, these controls do not isolate a single mechanism, and length-matched neutral-text and semantically-empty-trace controls are future work. We therefore *do not* claim counterexample-guided repair as a capability and treat these pairs as an exploratory data-format study; they are removed from the contribution list. A cautionary finding: over-weighting easy/medium safety-latch families drives easy-tier pass@1 to a perfect 1.000 but shifts the model *off* the genuinely hard multi-timer tasks (hard tier → 0). Data curation must match the target structural-complexity mix, not merely add "more hard-ish" families.

Best-of-ten provenance (Section VII-D of the main paper). The base and SFT best-of-ten numbers are fresh draws from a dedicated inference run (early-exit on the first task-valid program); the per-seed task-valid successes are base $[0.182, 0.227, 0.182] \rightarrow 0.197$ and SFT $[0.364, 0.273, 0.273] \rightarrow 0.303$, whereas RL-lite's best-of numbers reuse its standard evaluation samples. Early exit makes the call budget empirical rather than the 10 worst case: the mean best-of-ten oracle-call counts are SFT 7.651±0.022 (per-seed 7.682/7.636/7.636) and base 9.030±0.167 (`results/bestofk_call_counts.json`), so a single RL-lite call is 1/7.651=0.131 of the SFT best-of-ten empirical budget.

TABLE S-VIII
MODEL-SIZE SCALING OF VERIFIER-GROUNDED TRAINING (COMPILE-AND-INVARIANT PASS@k, THREE-SEED MEAN±STD). FROM THE ORIGINAL FULL-SUITE RUN SO THE THREE SIZES ARE ONE CONSISTENT SAMPLE; THE 7B PASS@1 VALUES THEREFORE DIFFER FROM THE ROUND-3 RE-SCORING (TABLE S-VI) BY ≤0.011 AND PASS@10 BY ≤0.024. ALL THREE SIZES USE THE 3-EPOCH SFT REGIME, WHOSE ENTROPY STATE DIFFERS FROM THE ENTROPY-PRESERVING 7B RESULT (TABLE S-VII); THE CROSS-SIZE RL COMPARISON IS THEREFORE NOT ENTROPY-CONTROLLED AND INDICATIVE ONLY.

Size	Stage	pass@1	pass@10
1.5B	base	0.000±0.000	0.000±0.000
	SFT	0.291±0.029	0.485±0.057
	SFT→RL	**0.364±0.046**	**0.621±0.057**
7B	base	0.068±0.007	0.349±0.057
	SFT	0.314±0.033	0.440±0.057
	SFT→RL	**0.344±0.008**	**0.455±0.037**
14B	base	0.070±0.013	0.288±0.021
	SFT	0.486±0.022	0.530±0.021
	SFT→RL	**0.491±0.028**	**0.561±0.043**

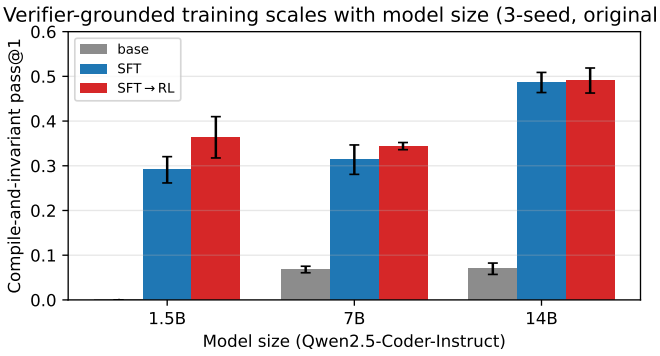


Fig. S-3. Procedural-domain SFT pass@1 grows with model size (three-seed, error bars are std); GRPO's marginal gain is largest at small scale and with a high-entropy warm-start. All sizes use the 3-epoch SFT regime, so size is not the only changing factor—entropy state and dataset exposure differ; treat as indicative, not a controlled scaling law. A 1.5B model with zero verified samples zero-shot reaches pass@1 0.364.

S.VIII. BENCHMARK TASK INVENTORY

Table S-XI lists all 22 tasks with their tier, number of formal safety properties, scenario-suite size, and principal ST constructs, totaling 71 properties (18 easy, 36 medium, 17 hard) and 48 execution scenarios (139 total scan steps; 1–3 scenarios per task). Every listed property is a safety/invariant LTL formula and every reference solution both compiles (MATIEC) and verifies (nuXmv).

TABLE S-IX
REPAIR FEEDBACK-CHANNEL CONTROLS (THREE TRAINING SEEDS, TASK-VALID PASS@1 ON A HELD-OUT BUG TYPE). ALL VARIANTS SHARE THE BUGGY ATTEMPT AND DIFFER ONLY IN THE FEEDBACK. THE FULL-COUNTEREXAMPLE-TRACE VARIANT DOES NOT OUTPERFORM THE NO-COUNTEREXAMPLE OR MISMATCHED-TRACE VARIANTS.

repair training feedback	pass@1
untrained base	0.388 ± 0.008
generic SFT (no repair pairs)	0.461 ± 0.006
no counterexample	0.553 ± 0.033
property only	0.592 ± 0.040
error message only	0.594 ± 0.025
full counterexample trace	0.514 ± 0.042
shuffled (mismatched) trace	$\mathbf{0.680 \pm 0.027}$

TABLE S-X
SFT DATA-COMPOSITION ABLATION, COMPILE-AND-INVARIANT PASS@1 (THREE-SEED MEAN±STD). THE "+ 50 HARD INSTANCES (GENERATION-ONLY)" ROW IS THE SFT-HARDGEN (GEN.-ONLY+HARD) CONDITION OF TABLE S-V.

SFT data composition	pass@1
158 base instances (7 families)	0.311 ± 0.025
+ 50 hard instances (generation-only)	$\mathbf{0.398 \pm 0.034}$
+ hard + 161 repair pairs (v2)	0.395 ± 0.039

TABLE S-XI
THE 22-TASK DUAL-ORACLE BENCHMARK. "#P" IS THE COUNT OF FORMAL SAFETY PROPERTIES; "#S" THE NUMBER OF EXECUTION SCENARIOS AND "ST." THEIR TOTAL SCAN STEPS. TON = IEC ON-DELAY TIMER; CASE = STATE MACHINE.

Task	Tier	#P	#S	st.	Principal constructs
`E01` motor interlock	easy	2	2	4	latch, e-stop, priority
`E02` tank overflow	easy	3	1	2	hysteresis, bounded `INT`
`E03` airlock interlock	easy	3	2	3	mutual exclusion
`E04` conveyor permissive	easy	3	2	4	permissive, guard, latch
`E05` alarm latch	easy	2	2	4	latch, reset
`E06` heater over-temp	easy	3	2	3	override, safety cutout
`E07` pump dry-run	easy	2	1	3	override, combinational
`M01` motor start delay	med.	2	3	10	TON on-delay, e-stop
`M02` star–delta	med.	4	3	8	CASE, TON, interlock
`M03` ped. crossing	med.	4	2	12	CASE, multi-TON, mutex
`M04` tank fill/drain	med.	4	3	8	CASE, bounded `INT`
`M05` conveyor sequence	med.	3	3	7	CASE, TON, sequencing
`M06` burner purge	med.	4	2	9	CASE, multi-TON, safety
`M07` two-hand control	med.	2	2	8	anti-tie-down, latch
`M08` pump alternation	med.	3	2	7	edge-detect, duty/standby
`M09` gate limits	med.	4	3	5	CASE, limit switch
`M10` three-conveyor start	med.	4	2	7	CASE, multi-TON, ordering
`M11` dosing counter	med.	2	2	7	counter, bounded `INT`
`H01` intersection signals	hard	4	1	4	CASE, multi-TON, mutex
`H02` elevator door	hard	4	3	5	CASE, door interlock
`H03` batch reactor	hard	5	3	8	CASE, TON, multi-actuator
`H04` parking counter	hard	4	2	11	up/down counter, `INT`

S.IX. COMPUTE AND ENVIRONMENT

Toolchain provenance. The MATIEC front end was built from the `master`-branch snapshot of the `sm1820` GitHub fork (accessed Jun. 27, 2026)—a third-party fork of M. de Sousa's upstream MATIEC (bitbucket.org/mjsousa/matiec; mirror github.com/beremiz/matiec); the exact source archive used is included in the artifact as `toolchain/matiec_src.tar.gz` (SHA-256 `9f1d8378459b043f...`, full hash in `ENVIRONMENT.md`). Training used the TRL 1.7 series (group-standard-deviation reward scaling, no explicit KL term); the exact wheel on the rented training host was not preserved, so the zero-variance-advantage behavior relied on in the main paper is verified empirically by the fixed-prompt diagnostic of Sec. S.XIII rather than by a version pin. xAI documents that the retired `grok-3` slug is served by `grok-4.3` with `reasoning_effort=none` after May 15, 2026; Anthropic documents that from the Claude 4.6 generation dateless model IDs are pinned snapshots; Google documents dynamic thinking by default for Gemini 2.5; nuXmv's license is academic / non-commercial, prohibiting modification and redistribution.

All experiments ran on a single rented NVIDIA RTX 5090 (32 GB). The API-model evaluation requires no GPU (it queries public APIs and runs the CPU-only oracle). The full training suite—RL prompt selection, the canonical 7B pipeline, the entropy-preserving pipeline, the 1.5/7/14B scaling study, the data and repair ablations, the functional-reward GRPO run, the filtering ablation, the entropy diagnostics, the two extra seeds, and the repair-control training—completed unattended in roughly 35 GPU-hours (about 19 for the original suite and 16 for the round-3 experiments). Training used 4-bit QLoRA (bitsandbytes) with TRL's GRPO implementation (TRL 1.7); inference and training disable cuDNN and use eager attention to sidestep a wheel-level cuDNN symbol-loading bug on this Blackwell card (plain CUDA matmul and 4-bit quantization are unaffected).

Hyperparameters. SFT uses LoRA rank 16, α=32, dropout 0.05 on all attention and MLP projections, per-device batch 1 with gradient accumulation 8 (effective batch 8), max sequence length 1024, and learning rate 2×10^{-4} (canonical 3-epoch regime) or 1×10^{-4} (entropy-preserving 1-epoch regime). GRPO uses a group size of 8 completions per prompt, gradient accumulation 4, learning rate 1×10^{-5}, max completion length 512, the graded dual-oracle reward of Eq. (2) of the main paper, and up to 30 update steps when refining an SFT policy (SFT→RL) or 50 steps when training a fresh LoRA on the base model; RL prompts are the 42–47 held-out frontier prompts selected by graded-reward variance under the probed policy. Sampling for all evaluations is n=10 per task at temperature 0.8. Exact package versions, seeds, per-stage logs, and the selected-prompt lists will be in the released artifact.

Toolchain. The compiler oracle is MATIEC (`iec2c`, M. de Sousa), invoked as `iec2c -I <lib> -T <out>` on the standalone `PROGRAM` (no `CONFIGURATION` wrapper; the `-I` library is MATIEC's bundled IEC 61131-3 standard-function-block set); a candidate compiles iff `iec2c` exits zero, and non-fatal warnings are treated as compiling. The model checker is nuXmv 2.2.0, run per property with BDD-based invariant/LTL checking; an `unknown`/timeout verdict (wall limit per property) counts the property as not proven. The exact MATIEC commit hash, build flags, nuXmv invocation, per-property outputs, and resource limits are recorded in the artifact's `ENVIRONMENT.md`. Our tasks are deliberately small (bounded integers, single scan loop), so per-property model

checking is BDD-tractable and cheap relative to model inference; we log per-call oracle wall-times and the total oracle-call volume (SFT filtering, prompt selection, and every GRPO rollout) in the artifact rather than summarizing them here. We do *not* characterize how oracle cost scales to larger state spaces or to a corpus of tens of thousands of instances, where the model checker rather than the GPU could become the bottleneck; oracle-cost scalability is an open question distinct from the training-data scarcity discussed in Section VIII of the main paper.

Model access. The four API models were queried in June and July 2026 with the request identifiers `grok-3` (xAI), `claude-sonnet-4-6` (Anthropic), `gemini-2.5-flash` (Google), and `gpt-4o` (OpenAI) over each provider's official endpoint, temperature 0.8, provider-default top-p, no stop sequences, one user message and no system message, and a max-token budget of 2048 per completion. We did not explicitly set provider reasoning/thinking budgets (e.g. Gemini's dynamic thinking may have been active by default), so the panel is a comparison of provider *API configurations* during the access window, not an architecture-controlled or reasoning-controlled comparison. As noted in Section V of the main paper, requests to the retired xAI `grok-3` slug are served by `grok-4.3` with `reasoning_effort=none` after 15 May 2026; the `grok-4.3`† rows denote that configuration (the harness retained only the generated program text, so per-response provider identifiers were not captured for any provider). *OpenAI provenance.* The OpenAI completions were requested with the model slug `gpt-4o` during the access window 2026-07-19; the harness recorded only the generated code, so per-response OpenAI model identifiers were not retained. The prompt is a single zero-shot user message (specification + typed interface requesting one ST `PROGRAM`); the returned ST is extracted from the first fenced code block, and a response with no fenced block is parsed as-is and typically scored *parse_error*. Each API call is retried once on a transport error and otherwise recorded verbatim. Providers exposed only through dateless aliases (e.g. `gpt-4o`) are recorded with their access window. The artifact releases the raw responses of the single-sample panel, the June per-task n=10 success counts, and the complete July 2026 n=10 task-valid panel with the generated program text retained for every sample (`results/frontier_n10/`); the June n=10 programs themselves were not retained, which is why the task-valid panel is a separate access window (Section V-D of the main paper).

S.X. Artifact Checklist

Table S-XII enumerates the released components, their status, and where each will live in the artifact. The authors have prepared these components and populated the public repository (see Artifact Availability). We do not assert reviewers can access them from this manuscript alone.

S.XI. Supported Subset and Edge Cases

The translator front-end supports scalar `BOOL`/`INT` variables, `IF`/`ELSIF`/`CASE`, Boolean and comparison operators, bounded-integer arithmetic, and `TON` timers with whole-second presets. Explicit scalar variable initializers (e.g. `x : INT := 5`) are supported and honored as the model's initial state. Arrays, user-defined functions and function blocks, enumerations, `REAL`/floating-point, general `TIME` arithmetic, strings, `RETAIN`/persistent-variable qualifiers, and vendor-specific extensions are outside this subset and are rejected by the front-end. A valid IEC program that uses an unsupported construct is thus distinct from malformed ST, but both are scored *front-end* (never *verified*); in training they receive the lowest reward tier. This is deliberately conservative, charging every translator gap against the model and never in its favor, but it means a "front-end" rejection can reflect either invalid ST *or* merely an out-of-subset construct—a distinction we separate in the finer taxonomy of Section IV-B of the main paper. Importantly, MATIEC compiles the model's *literal* output; we do not rewrite the source before compilation. Case-insensitive identifier resolution is handled internally by the translator's symbol table (not by editing the program text), so the compiler and translator both see the original candidate. MATIEC warnings that do not abort compilation are treated as compiling: a candidate compiles iff `iec2c` returns success. `iec2c` is a front-end that parses and type-checks the ST and emits ANSI C; "compiles" in this paper therefore means *MATIEC front-end acceptance*: the program is accepted and translated by this front-end, not that a runnable binary was linked and executed on a target. Five successively stronger acceptance levels can be distinguished—(1) translator parseability, (2) MATIEC IEC front-end acceptance, (3) successful compilation of the emitted C by a pinned C compiler, (4) runtime execution on a target, and (5) vendor-toolchain acceptance—and this work establishes only levels (1)–(2); every use of "compiles" herein should be read as level (2), and adding level (3) is a natural future check. The compiler oracle is a real *open-source* IEC 61131-3 compiler front end and is not a substitute for a vendor toolchain (Siemens TIA Portal, CODESYS, TwinCAT, or Rockwell); vendor-deployable validity is a stronger bar we do not claim.

TABLE S-XII
Reproducibility checklist. Paths are relative to the artifact root; versions and commit hashes are recorded in `ENVIRONMENT.md`.

Component	Status	Location / version
22 tasks + 71 properties	prepared	`benchmark/tasks/`
Reference ST solutions	prepared	`benchmark/tasks/*/ref.st`
ST→SMV translator	prepared	`plcbench/st/smv.py`
Scan-cycle interpreter	prepared	`plcbench/st/interp.py`
MATIEC build (`iec2c`)	prepared	pinned commit in `ENVIRONMENT.md`
nuXmv	prepared	v2.2.0
pass@k + bootstrap scripts	prepared	`eval/passk.py`, `eval/boot.py`
Raw API responses (single-sample panel)	prepared	`results/*_*.json`
Per-task n=10 API success counts	prepared	`docs/source/passk_*/`
Raw July n=10 API completions (task-valid panel)	prepared	`results/frontier_n10/`
Differential translator validation	prepared	`analysis/difftest_translator.py`, `results/difftest/`
SFT corpus (158/369)	prepared	`finetune/data/`
RL prompt-selection data	prepared	`finetune/rl_prompts/`
Training/eval code + configs	prepared	`finetune/`
LoRA adapters (all stages)	prepared	`finetune/out/`
Per-seed result files	prepared	`results/full/`, `results/seeds/`

S.XII. Extended Threats-to-Validity Discussion

This section preserves the full discussion summarized in Section VIII of the main paper.

Time abstraction. We use a scan-indexed discrete-time abstraction with step period Δt=1 s (all benchmark presets are

whole seconds), a modeling choice for state-space tractability in nuXmv; as defined in Section IV of the main paper, each model transition is a *logical* time step—a one-second macro-step aggregating many physical PLC scans—not a physical scan. This is honest for *logical* timing and sequencing (interlock ordering, elapsed-step counting, mutual exclusion) but is deliberately blind to sub-step phenomena: a 3 s preset at a 10 ms real scan is $\sim$300 physical scans compressed into 3 macro-steps, so millisecond-scale race conditions, scan-time jitter, wall-clock-latency hazards, and input changes within the physical scans omitted inside a macro-step are *out of scope*. Our modeled result that a timer's done bit becomes true on the P-th continuous logical step is a property of this abstraction; conformance to IEC/MATIEC/runtime timer semantics at the exact activation boundary is checked only against our interpreter, not a vendor runtime. We therefore label timing results as *scan-step timing under the benchmark abstraction*; a configurable Δt and a timed-automata back-end are natural extensions. *Environment assumptions.* Inputs are modeled as free (unconstrained) SMV variables, so verification quantifies over *all* input traces. This is conservative *only with respect to the modeled invariants*: physically impossible input combinations (e.g. mutually exclusive limit switches both true) can produce *false counterexamples* that penalize otherwise-correct logic, and we encode no environment or fairness assumptions, restricting the benchmark to safety (invariant) properties; task-level environment assumptions (assume-guarantee) and liveness reasoning are future work. Accordingly, the property suite is one-sided—69 of 71 invariants are prohibition-style (Section V-C of the main paper)—so the benchmark should be read as a safety-prohibition benchmark augmented with finite functional traces; formal positive obligations (bounded response, eventual progress under stated environment assumptions) are the corresponding future work. A subtler consequence is that, because the oracle assumes in-range inputs, the oracle-checked SFT data it produces contains no defensive/out-of-bounds handling—so training on it implicitly rewards trusting inputs and may suppress defensive programming. Inducing robust code would require modeling adversarial or noisy inputs in the oracle, which we leave to future work. *Integer semantics.* The bounded-range rejection is not a translator-side analysis: the translator performs no interval analysis, symbolic range propagation, or SMT solving. It requires a declared $[\min, \max]$ range for every `INT` (a program with an unbounded integer is rejected outright), declares each integer state variable as an SMV bounded-subrange type, and emits the end-of-scan update expression unchecked; an assignment that could evaluate outside the declared range is then rejected by *nuXmv* at model-construction time, which the harness surfaces as *translate_error* (scored 0.0)—it does not clamp, wrap, or drop transitions. This delegated check is conservative against the candidate (any rejection or inconclusive outcome withholds *verified*); its precision on guarded arithmetic is nuXmv's, not ours—the benchmark's guard-protected counter references are accepted—and we have not characterized it beyond that. The benchmark's counter tasks are written so that every increment/decrement is guarded against the bound, so this case does not arise for verified references. We do not model MATIEC's own integer behavior (it lowers `INT` to a fixed-width C type whose overflow follows that type's semantics); characterizing that divergence experimentally is future work. This is a benchmark modeling choice, not a claim about IEC integer semantics. *External validity and overfitting.* Training gains are demonstrated on Qwen2.5-Coder at three sizes with QLoRA and GRPO. The SFT corpus is small and templated (158–369 procedural instances from a fixed family grid); a model fine-tuned on it risks overfitting to that grid, and the reported pass@k may degrade on industrial logic structurally unlike our families. The entropy collapse we diagnose is itself partly a symptom of this scale (three epochs suffice to memorize so few templates). Two features partly mitigate a pure-memorization reading, without settling it. First, the 22 evaluation tasks are hand-authored, not emitted by the SFT generator, and the hard tier in particular (`H01` four-way intersection, `H02` elevator-door interlock, `H03` batch-reactor sequence, `H04` up/down parking counter) combines constructs that no single training family produces, so the scaled-SFT gains there are not explained by lexical template reuse alone. Second, the ablation in which over-weighting easy/medium latch families drives easy-tier pass@1 to 1.000 yet collapses the hard tier to 0 shows the model does *not* transfer for free across construct types—consistent with genuine, if narrow, learning rather than uniform memorization. A controlled family-level holdout and an out-of-distribution industrial benchmark (with SFT loss/generalization curves) are the decisive test and remain future work. Confirming the method transfers to other open families (e.g. DeepSeek-Coder, StarCoder2), to full-parameter training, and to larger and more heterogeneous corpora, and analyzing which *property types* (timing vs. combinational vs. counting) are hardest, are further natural extensions.

S.XIII. Entropy Diagnostics, Prompt Selection, and the 7B Ladder

This section gives the measurement protocols summarized in Section VI-C of the main paper, and plots the full 7B training ladder (Fig. S-4).

Prompt selection. For RL we select 42–47 training prompts by graded-reward *variance* under the probed policy, computed as follows. We hold out the four harder *training* families (never the 22 evaluation tasks), sample the policy under evaluation (k completions per candidate prompt at temperature 0.8), score each completion with the dual oracle, and keep the prompts whose reward standard deviation is highest—i.e. prompts the current policy half-solves, which carry the most gradient. Selection uses only held-out training-family outcomes, never the benchmark; it is redone for each policy being trained (SFT-lite, base). This is a training-time curriculum choice, not a benchmark-tuning step. We report the entropy-collapsed and entropy-preserving pipelines side by side because the contrast is the methodological lesson.

A fixed-prompt diagnostic ties the RL gain to warm-start entropy. To reduce the confound between the two warm starts (which differ in epochs, learning rate, *and* entropy), we ran GRPO from each warm start on the *same* fixed 39-prompt frontier set with the same reward and rollout budget,

logging per-step diagnostics. The entropy-preserving SFT-lite warm start (mean entropy 0.151) leaves only 0.10 of GRPO groups with zero within-group reward variance, whereas the collapsed 3-epoch SFT (entropy 0.015) leaves 0.775 zero-variance—i.e. most groups carry no gradient. (The entropies quoted in this diagnostic are measured on the fixed 39-prompt frontier set; the $\approx$ 0.001 reported above is the same 3-epoch SFT measured on its own training prompts—the two values differ in measurement prompt set, not in checkpoint.) Adding an explicit entropy bonus to the collapsed policy barely moves either quantity ($0.775 \rightarrow 0.75$), so a small regularizer cannot rescue an already-collapsed policy. On the same fixed protocol, across the two SFT warm starts the downstream task-valid pass@1 is monotonic in warm-start entropy (0.215 at 0.015 nats, 0.282 at 0.151); GRPO from the untrained base, with no warm start, reaches only 0.052 despite its higher sampling entropy ($\approx$ 0.20 nats measured on this same set), so it is a no-warm-start floor rather than a third point on the entropy ordering. This strengthens the association—under this fixed protocol, higher warm-start entropy was associated with fewer zero-variance GRPO groups and, among the SFT warm starts, higher downstream pass@1—while we still avoid a strict causal claim, since matched-validation checkpoints along a single SFT trajectory would be needed to fully isolate entropy from training length.

A structural caveat on what "compile-and-invariant" rewards. Our criterion is compilation plus *safety-invariant* satisfaction; it contains no liveness or functional requirement. A degenerate program such as `Motor := FALSE;` compiles and trivially satisfies most safety invariants (a motor that never energizes cannot violate mutual exclusion or an e-stop cutoff), so it would score compile-and-invariant while solving no task. The reference solutions are additionally checked against input/expected-output scenarios, but those scenarios are *not* part of the compile-and-invariant criterion or of the invariant-only reward of Eq. (2) of the main paper; the functional reward of Eq. (3) of the main paper is the alternative that does include them. This means "compile-and-invariant" should be read as *compile-and-safety-property pass rate*, not "task solved" or "correct PLC code," and a policy could in principle reduce reward-seeking to producing inert but safe programs. We explicitly *withdraw* the intuitive argument that improving every tier argues against it: that argument is invalid, because the all-outputs-`FALSE` program of Section V-C of the main paper is itself compile-and-invariant on *all* 11 medium and *all* 4 hard tasks (it fails only two easy tasks). Deciding the question requires controls, which we now run rather than defer: per-condition all-off/all-on/constant baselines and a behavioral output-activity probe (Table S-IV), and a task-valid acceptance metric and RL reward that require functional-scenario success (Section V-D of the main paper, Eq. (3) of the main paper). The probe's quantitative outcome is reported once, in Section VI-B of the main paper: activity rises along the tested progression and no drift toward inactivity is detected, though the shaped reward of Eq. (2) of the main paper still does not preclude it by construction. Liveness obligations beyond the current bounded scenarios remain future work; critical-property gating is now evaluated (Sec. S.XVIII: no all-checks-pass cost, a small seed-consistent compile-and-invariant gain).

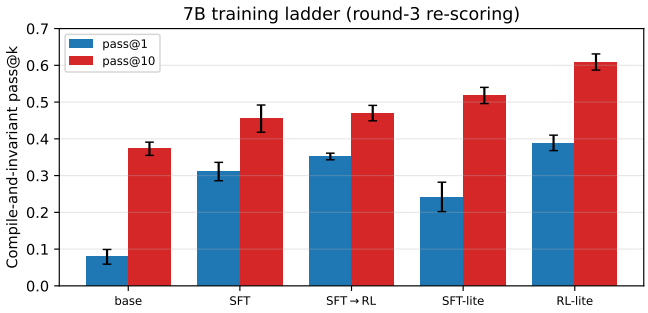


Fig. S-4. 7B training ladder (five training seeds for base/SFT-lite/RL-lite, three otherwise; error bars are across-seed std). A 3-epoch SFT already lifts base substantially; the entropy-collapsed 3-epoch warm start was associated with a much smaller subsequent RL gain. Warm-starting RL from the entropy-preserving 1-epoch SFT-lite gave the largest RL gain and the best-scoring RL condition shown (RL-lite, 0.389 ± 0.021); RL-lite remains below the strongest SFT condition (0.398, Table S-X), and the later gated-reward run reaches 0.411 ± 0.008 (Table S-XIV)—gaps this seed count does not resolve.

S.XIV. Statistical Detail for the API Panel

Primary metric (July all-checks-pass panel, n=10). Paired task bootstrap, 20,000 resamples, seed 0. Per-model pass@1 (95% CI): grok-4.3† 0.591 $[0.455, 0.727]$; claude 0.459 $[0.264, 0.668]$; gemini 0.100 $[0.032, 0.182]$; gpt-4o 0.095 $[0.023, 0.191]$. Pass@10: 0.909 $[0.773, 1.000]$; 0.500 $[0.273, 0.727]$; 0.273 $[0.091, 0.455]$; 0.364 $[0.182, 0.591]$ (pass@10 is effectively a 22-task binary any-success statistic; treat with caution). Pairwise pass@1 differences: grok$-$claude $+0.132$ $[-0.009, +0.282]$; grok$-$gemini $+0.491$ $[+0.373, +0.614]$; grok$-$gpt-4o $+0.495$ $[+0.336, +0.645]$; claude$-$gemini $+0.359$ $[+0.200, +0.532]$; claude$-$gpt-4o $+0.364$ $[+0.195, +0.545]$; gemini$-$gpt-4o $+0.005$ $[-0.073, +0.077]$. A hierarchical variant that also resamples completions within tasks widens the intervals only modestly (e.g. grok $[0.441, 0.736]$). All six contrasts are unadjusted and exploratory. Per-task all-checks-pass counts are released in the artifact (`results/frontier_n10/`).

RQ1 metric (July compile-and-invariant panel, n=10). Same protocol (20,000 resamples, seed 0; `analysis/ci_bootstrap_july.py`). Per-model pass@1 (95% CI): grok-4.3† 0.677 $[0.532, 0.814]$; claude 0.555 $[0.355, 0.755]$; gpt-4o 0.109 $[0.036, 0.205]$; gemini 0.100 $[0.032, 0.182]$. All four cross-tier contrasts exclude zero (grok$-$gpt-4o $+0.568$ $[+0.386, +0.736]$; grok$-$gemini $+0.577$ $[+0.441, +0.714]$; claude$-$gpt-4o $+0.445$ $[+0.259, +0.636]$; claude$-$gemini $+0.455$ $[+0.282, +0.627]$), while both within-pair contrasts include zero (grok$-$claude $+0.123$ $[-0.027, +0.286]$; gpt-4o$-$gemini $+0.009$ $[-0.068, +0.091]$); a hierarchical variant that also resamples completions within tasks widens the intervals only modestly (e.g. grok $[0.523, 0.818]$). This is the analysis cited by RQ1 of the main paper.

Secondary (June compile-and-invariant panel). Because every model is tested on the same tasks, we assess differences with a *paired* bootstrap over task-wise pass@1

differences (20,000 resamples; $\Delta_{A,B} = \text{pass@1}(A) - \text{pass@1}(B)$). The two stronger models each beat each weaker model with intervals excluding zero (e.g. claude−gpt-4o $\Delta{=}{+}0.473$, 95% CI $[+0.291, +0.650]$; grok−gemini $+0.550$ $[+0.418, +0.686]$), whereas the two within-pair contrasts do *not* exclude zero (grok−claude $+0.082$ $[-0.050, +0.232]$; gemini−gpt-4o $+0.005$ $[-0.118, +0.118]$). Because each per-task rate is itself estimated from only 10 completions, we also ran a *hierarchical* bootstrap that resamples tasks and then completions within each task; this widens the intervals only marginally (e.g. grok pass@1 $[0.486, 0.795]$ vs. $[0.495, 0.791]$; claude−gpt-4o $[+0.291, +0.655]$), because task-to-task variance dominates, and it leaves the conclusion unchanged. Full bootstrap details and the complete six-pair comparison matrix are released in the artifact; because the four models yield six pairwise contrasts and we apply no multiplicity adjustment, this pairwise analysis is exploratory. We therefore claim only a two-tier separation and make no grok-vs-claude or gemini-vs-gpt-4o ordering.

S.XV. Vacuity and Mutation Analysis

Why scenarios add discriminating power. We quantify this two ways. (i) *Property vacuity*: of the 71 properties, 69 (0.972) are satisfied by the inert all-off controller; only 2 are positive-output obligations. The forms are guard-implies-off (30), mutual-exclusion (13), output-implies-precondition (24), input-only (2, e.g. `G(Count<=3)`), and positive-obligation (2)—a one-sided suite that an inactive program passes vacuously. (ii) *Mutation analysis*: we generated syntactic mutants of the reference solutions by deterministically and exhaustively applying seven operator families at every applicable site of every reference (Boolean-literal flips, comparison-operator swaps, `AND`/`OR` connective swaps, `IF`/`ELSIF` condition negation, single-assignment deletion, timer-preset off-by-one, and comparison-constant off-by-one—no sampling and no random seeds; per-task counts in the artifact), and kept the 384 mutants that still compile under MATIEC. Invariants kill 172 (0.448), scenarios kill 268 (0.698), and either kills 294 (0.766); crucially, 122 mutants (0.318) pass *all* invariants yet fail a scenario—faults the invariant-only metric cannot see but the scenario dimension catches. (This mutation battery also exercises the ST→SMV translator on 384 programs, well beyond the one hand-injected mutation per task used for its initial validation.) We say *exercises* rather than *validates*: the same translator and interpreter grade every mutant, so the battery demonstrates sensitivity to injected faults, not semantic correctness—a semantic error shared by the translator and the grading pipeline would remain invisible here. The remaining 90 mutants (0.234) survive both dimensions and include probable equivalent mutants, an inherent caveat of mutation testing. As an invariant-by-scenario 2×2 over the 384 compiling mutants:

	scenario kills	scenario survives
invariant kills	146	26
invariant survives	122	90

The 122 scenario-only kills ($122/384 = 31.8\%$) are exactly the invariant-blind faults. Per-mutant outcomes were not persisted; this 2×2 is fully determined by the saved per-task aggregates (`results/mutation_2x2.json`, re-verified) and reconciles with the margins above ($146{+}26{=}172$ invariant kills, $146{+}122{=}268$ scenario kills, 90 survivors).

Scenario-count sensitivity. The all-scenarios-pass verdict is robust to scenario-count reduction: on the July panel, dropping the single most influential scenario changes the verdict for 8 of 880 samples (the maximum, `M10`), the median single-scenario removal changes none, and only $15/880$ samples fail exactly one scenario (`results/scenario_sensitivity.json`).

S.XVI. Exact Category-to-Reward Mapping

This section states the exact mapping from the eight harness categories of Section IV-B of the main paper to the training rewards. The reward is keyed on translator-parseability first. The parser (`parse_program`) is the 0.0 gate: *no-program* and *parse_error* (and any out-of-subset *unsupported* construct the parser cannot accept) score 0.0, as does an uncaught oracle exception (the arbitrary-bytes/subprocess guard, *api_error* included). A parseable program that MATIEC rejects (*compile_error*) scores 0.1. Crucially, a program MATIEC *compiles* but the translator cannot lower to SMV—harness category *translate_error*, e.g. an unranged integer rejected at model construction—is *not* 0.0: it reaches the graded branch with every property counted unproven and scores $0.3 + 0.7 \cdot 0 = 0.3$ under Eq. (2) (correcting an earlier draft that stated 0.0; verified end-to-end against the released reward implementation, `analysis/reward_translate_error_e2e.py`).
A *timeout*/`unknown` nuXmv verdict likewise leaves the affected property unproven *inside* the graded branch, never 0.0, so *verified* $= 1.0$ and *unsafe* < 1.0. The full map is below (columns: Eq. (2) dual-oracle reward, Eq. (3) task-valid reward, and the critical-gated variant of Sec. S.XVIII; "graded" = property/scenario-dependent; source `results/reward_category_table.json`):

harness category	Eq. (2)	Eq. (3)	gated
parse_error (incl. no-program, unsupported)	0.0	0.0	0.0
compile_error	0.1	0.1	0.1
translate_error (compiles, un-lowerable)	0.3	graded	0.1
verified	1.0	1.0	1.0
unsafe, non-critical property fails	graded	graded	graded
unsafe, critical property fails	graded	graded	0.1
verify *timeout* (property unproven)	graded	graded	graded
uncaught exception (incl. *api_error*)	0.0	0.0	0.0

Scenario dimension (Eq. (3) only). The table keys on the static/formal categories; the functional term enters only through Eq. (3) of the main paper. A mid-scenario expected-output mismatch or an interpreter runtime error zeroes only the s_{pass}/s term of Eq. (3) (leaving the compile and property terms intact); Eq. (2) has no scenario term, and the gated variant additionally caps the total at 0.1 whenever a critical property is unconfirmed. Both this table and the scenario-term behavior are emitted directly from the released reward implementation (`analysis/reward_category_table.py` and `analysis/reward_translate_error_e2e.py`), so the paper and the harness cannot drift.

An honest exposure of this: in the 200-program differential-test model pool, 44 were *translate_error* and would each earn 0.3 under Eq. (2)—a disclosed imprecision of the training *reward*, not of the evaluation *metric*, which counts *translate_error* as not-verified (it never contributes to a compile-and-invariant or all-checks-pass pass). Precedence is translator-parse $\rightarrow$ MATIEC-compile $\rightarrow$ property-check. Every task has $p{>}0$; if a completion contains multiple programs we take the first `PROGRAM` block. *All-off reward caps.* Measured over the 22 tasks, the inert all-off controller earns mean reward 0.973 (max 1.000) under Eq. (2) but only 0.642 (max 0.867) under Eq. (3)—a residual shaping limitation of the task-valid reward. Sampling for GRPO rollouts uses temperature 1.0; all training runs use three training seeds, except base, SFT-lite, and RL-lite, which use five (Table III of the main paper).

S.XVII. Timer Trace Tables, T#1s Exposure, and the Read-Before-Call Defect

Per-scan TON traces (unchanged literal source). The driver sets the wall clock to $k \cdot \Delta t$ before the FB call of scan k. For `IN` held TRUE from scan 1: with $P{=}1$, C asserts `Q` at scan 2 while the interpreter/SMV recurrence asserts it at scan 1; with $P{=}2$, scan 3 vs. scan 2; with $P{=}3$, scan 4 vs. scan 3—uniformly one scan later in C, because MATIEC records `START_TIME` at the rising scan and requires elapsed $\geq$`PT` (`ET` $= 0$ on the rising scan), whereas the tick abstraction counts the rising scan as one elapsed step. The full tables (including release and re-rise behavior, C `START_TIME`/`ET` internals, and the interpreter counter) are generated by `analysis/ton_trace_table.py` and included in the artifact (`results/difftest/ton_trace_table.txt`). Rewriting the preset literal of the source given to MATIEC by one tick aligns the two exactly for all $P \geq 2$; for $P{=}1$ no wall-clock preset can assert `Q` on the rising scan, so 1-tick presets are an inherent boundary divergence of the abstraction.

T#1s exposure audit. One of the 22 references uses 1-tick presets (`H01`, two of its three timers); 8/158, 16/369, and 8/208 SFT examples, 548/6,600 trained-model completions, and 32/880 July-panel completions contain `T#1s`. For `H01`, all four safety invariants were additionally evaluated on the MATIEC-C wall-clock traces of the differential test (200 scans): zero violations—the one-scan phase shift changes when transitions occur, not the verified mutual-exclusion/clearance structure, on the sampled traces. Under the abstraction's stated reading (each scan represents one second of held input), the tick semantics remain self-consistent; results on `T#1s`-dependent programs are scan-granularity–relative, as flagged in Section VIII of the main paper.

Read-before-call defect and repair. The SMV trace leg initially failed on 14 of 300 grammar-generated programs. Diagnosis: the translator pre-seeded a read of a timer's `Q` occurring *before* that timer's call in scan order with constant `FALSE`, whereas the interpreter yields the previous-scan value. The fix pre-seeds `Q` with (`ET_prev` $\geq P$), which equals the previous-scan `Q` for $P \geq 1$ (the preset is prescanned from the top-level call). After the fix the SMV leg proves the expected trace on every reference and grammar-generated program. Blast radius: no reference, SFT example, or June-panel program contains the construct; 52/6,600, 10/660, and 5/880 evaluated samples do, and re-scoring all of them under the fixed translator changed *zero* invariant or all-checks-pass verdicts (`results/difftest/qbeforecall_rescore.json`). Presets below one tick (e.g. `T#500ms`, which rounds to 0) remain accepted with their legacy translation for compatibility with previously scored programs; no evaluated program combines a sub-tick preset with a read-before-call.

TABLE S-XIII
FILTERING ABLATION (THREE SEEDS, TASK-VALID PASS@1); FIVE MATCHED SFT SETS FROM ONE POOL OF 632 BASE-7B SAMPLES (88 PASSED BOTH ORACLES; OF THE 544 REJECTED, 404 PASSED COMPILATION ONLY, 15 PROPERTY CHECKING ONLY, 125 NEITHER). THE SIZE-MATCHED RANDOM CONTROL SEPARATES THE CRITERION FROM CORPUS SIZE.

SFT trained on	size	task-valid pass@1
all (unfiltered)	632	0.014±0.004
MATIEC-accepted (incl. dual-pass)	492	0.020±0.009
property-passing (incl. dual-pass)	103	0.086±0.013
dual (compile ∧ property)	88	**0.127**±0.010
random, size-matched to dual	88	0.050±0.010

Semantics-sensitivity audit. To quantify whether any verdict depends on the timer first-assertion boundary, every evaluated sample containing `T#1s` was re-scored under a delayed-`Q` variant (`Q` first true on the $(P{+}1)$-th continuous-`IN` scan, matching unchanged-source MATIEC behavior; variant validated by unit traces and nuXmv cross-checks before scoring). Of 666 units: trained-model evaluation (exp1a) 548 samples, 4 invariant / 18 scenario / 7 all-checks-pass verdicts change; functional-reward runs 34 samples, 0/0/0; July open panel 32 samples, 0/6/5; July constrained panel 51 samples, 0/26/20; and the `H01` reference itself, whose scenario is authored against tick semantics and fails under the variant (0/1/1). Totals: 4 invariant, 51 scenario, 33 all-checks-pass changes. Invariant verdicts are thus nearly boundary-independent; the sensitivity is concentrated in the scenario dimension, whose expected outputs encode exact per-scan timing. Samples not reaching a clean verdict on both legs (unparseable output, unranged integers) are conservatively excluded. Raw results: `results/t1s_boundary_audit.json`.

Sampled trace coverage. Over the executed differential-test traces, the sampled coverage was 1,690/2,487 `IF` branches and 918/1,302 `CASE` labels; of 342 timers, 144 exercised rise, expiration, and reset, while 89 never rose, 8 rose without expiring, and 101 expired without resetting (per-event breakdown in the artifact). This is sampled evidence, not exhaustive semantic validation.

S.XVIII. Full Training Tables

Table S-XIII is the filtering ablation summarized in Section VII-D of the main paper; Table S-XIV is the full training table behind main-paper Table III, with compile-and-invariant columns.

TABLE S-XIV
ALL-CHECKS-PASS (TASK-VALID) AND COMPILE-AND-INVARIANT ("VERIFIED") PASS@k OF THE TRAINED MODELS: MEAN OVER INDEPENDENT SEEDED RUNS (FIVE FOR BASE/SFT-LITE/RL-LITE, THREE OTHERWISE), n=10 PER TASK, TEMPERATURE 0.8; SD IS ACROSS TRAINING SEEDS (POPULATION SD). SAME ROUND-3 RUN AS TABLE S-VI. EVERY ROW BEATS THE INERT BASELINE; SFT-V2 REMAINS THE STRONGEST CONDITION ON ALL-CHECKS-PASS; ON COMPILE-AND-INVARIANT THE GATED-REWARD RL ROW IS NOMINALLY HIGHEST (0.411, VS. THE HARD-FAMILY SFT 0.398 OF TABLE S-X AND SFT-V2 0.395)—DIFFERENCES THREE SEEDS DO NOT RESOLVE.

	task-valid		verified	
condition	pass@1	pass@10	pass@1	pass@10
base	0.045±0.018	0.218±0.053	0.079±0.020	0.373±0.018
SFT (3-epoch)	0.186±0.007	0.288±0.021	0.311±0.025	0.455±0.037
SFT→RL	0.197±0.017	0.273±0.000	0.352±0.009	0.470±0.021
SFT-v2	**0.362**±0.026	0.394±0.021	0.395±0.039	0.470±0.057
SFT-lite	0.179±0.028	0.409±0.050	0.242±0.040	0.518±0.022
RL-lite	0.272±0.011	**0.427±0.022**	0.389±0.021	0.609±0.022
RL (func. reward)	0.282±0.010	0.409±0.000	0.382±0.004	0.591±0.037
RL (crit. gated)	0.292±0.015	0.424±0.021	**0.411**±0.008	**0.652±0.021**
all-off baseline	**0.000**	0.000	—	—

TABLE S-XV
MATCHED-SEED ALL-CHECKS-PASS PASS@1: SFT-LITE WARM START VS. ITS FUNCTIONAL-REWARD RL RUN (SAME TRAINING SEED; n=10).

seed	SFT-lite	RL (func.)	difference
0	0.182	0.295	+0.114
1	0.127	0.277	+0.150
2	0.214	0.273	+0.059
paired mean +0.108, SD 0.037			

On the ±0.000 *pass@10 entries.* These denote exact seed agreement, not rounding: SFT→RL reaches task-valid pass@10 on $6/22$ tasks in all three training seeds (0.2727 each), and RL (func. reward) on $9/22$ tasks in all three (0.4091 each), so the across-seed standard deviation is genuinely 0.000.

Matched-seed paired comparison. The functional-reward RL runs warm-start from SFT-lite seeds 0–2, so the seed-matched comparison isolates the RL effect (Table S-XV). All three paired differences are positive (mean +0.108, SD 0.037); the three-seed matched SFT-lite baseline is 0.174±0.036 (five-seed estimate 0.179 ± 0.028), and the RL runs' across-seed SD (0.010) is reported alongside rather than compared across unequal seed counts. All across-seed and across-pair SDs are population SDs (ddof=0).

Critical-property-gated reward. As a safety-oriented reward variant, we hard-gate the functional reward (Eq. (3) of the main paper) to $\min(r_{\text{tv}}, 0.1)$ whenever any *critical-severity* property (36 of the 71) is not confirmed passing, so a completion that misses a critical interlock cannot bank compile/scenario credit. Warm-started from the same SFT-lite seeds 0–2 under the identical 50-step GRPO protocol as the functional-reward run (num_gen 8, grad_accum 4, temperature 1.0, same 39-prompt frontier set), the gate leaves all-checks-pass essentially unchanged—paired mean +0.011 pass@1 (SD 0.019; per-seed $-0.009/+0.036/+0.005$, mixed signs, within noise) and +0.015 pass@10—while yielding a small but seed-consistent gain on compile-and-invariant compliance: paired mean +0.029 pass@1 (SD 0.004; positive in all three seeds, $+0.032/+0.032/+0.023$) and +0.061 pass@10 (SD 0.057, noisy). Its compile-and-invariant pass@1 (0.411, added row of Table S-XIV) is the highest among the trained conditions, but at a lower all-checks-pass rate than SFT-v2, so it does not change the strongest-system statement (which is scoped to all-checks-pass). Training reward rose across all three seeds (first-five-step to last-five-step mean $0.39 \to 0.66$, $0.23 \to 0.78$, $0.35 \to 0.69$); the gated reward function passed $9/9$ unit tests on the run VM and the oracle was sanity-checked on reference tasks before the run (exact environment recorded in `results/exp_gated/ENV.txt`: TRL 1.8.0, transformers 5.14.1, torch 2.11.0+cu128; the functional-reward baseline ran on the original training host, TRL 1.7 series with the exact wheel not preserved, so the paired comparison crosses library versions as well as reward definitions). At three seeds this supports no safety-significance claim beyond the stated paired means.

TABLE S-XVI
OPEN VS. HARNESS-CONSTRAINED PROMPT MODES (JULY 2026, n=10, TEMPERATURE 0.8; PASS@1 EXCEPT LAST COLUMN). ACP = ALL-CHECKS-PASS. †GROK-4.3, REASONING_EFFORT=NONE, VIA THE RETIRED GROK-3 SLUG.

Model	Mode	compile	comp.+inv.	scenario	**ACP**	ACP@10
grok-4.3†	open	0.895	0.677	0.673	**0.591**	0.909
	constr.	0.905	0.873	0.864	**0.755**	1.000
claude-sonnet-4-6	open	0.923	0.555	0.473	**0.459**	0.500
	constr.	0.873	0.791	0.755	**0.668**	0.773
gemini-2.5-flash	open	0.127	0.100	0.200	**0.100**	0.273
	constr.	0.068	0.068	0.145	**0.068**	0.273
gpt-4o	open	0.141	0.109	0.286	**0.095**	0.364
	constr.	0.264	0.200	0.432	**0.145**	0.364

S.XIX. HARNESS-CONSTRAINED PROMPT MODE

Item: does the open-mode panel penalize models for guessing the harness's accepted subset? We re-queried all four providers (July 2026, n=10, temperature 0.8, all generated program text retained for every sample, `results/frontier_n10_constrained/`) with the same prompt plus an explicit statement of the accepted subset (scalar `BOOL`/`INT`; `IF`/`CASE`; `AND`/`OR`/`NOT`, comparisons, `+`,`−`,`*`,`MOD`; `TON` only, called once, unconditionally, at top level, with whole-second literal presets; no arrays, `REAL`, strings, user FBs, other standard FBs, `RETAIN`, or direct addressing; exact wording in the artifact). Table S-XVI compares the modes. Disclosure substantially raises the two stronger configurations (claude all-checks-pass pass@1 +0.209; grok-4.3† +0.164, reaching pass@10 1.000), modestly raises gpt-4o (+0.050), and slightly lowers gemini (−0.032)—consistent with the open mode's documented bias *against* subset-capable models, while weaker models gain little or lose (longer instructions without the capability to follow them). Model comparisons in the main paper remain open-mode (zero-shot, no harness knowledge); the constrained mode bounds the hidden-subset penalty rather than replacing the primary protocol.

TABLE S-XVII
AUTHOR-SIDE EXPERT REVIEW OF THE 22-TASK BENCHMARK: PER-DIMENSION COUNTS OF ACCEPTABLE/MINOR-ISSUE/DEFECT OVER 22 TASKS (ONE SCORE PER TASK PER DIMENSION; 220 CELLS TOTAL). VERDICT SUMMARY: 1 ACCEPT, 18 ACCEPT-WITH-FIXES, 3 REJECT-LEVEL; 7 TASKS CARRY A BLOCKING DEFECT. ALL 22 REFERENCES REMAIN MACHINE-GREEN UNDER THE V1.0 SUITES; FINDINGS ARE RELEASED AS ERRATA (TEXT BELOW).

	Dimension	Accept.	Minor	Defect
D1	Requirement realism	12	10	0
D2	I/O polarity correctness	20	2	0
D3	Scan-cycle behavior	19	2	1
D4	Timer semantics / presets	19	2	1
D5	Fault / restart handling	15	5	2
D6	Safety-property completeness	9	8	5
D7	Property correctness (LTL/NL)	20	1	1
D8	Scenario coverage	10	10	2
D9	Reference-solution correctness	18	1	3
D10	Tier assignment	18	4	0
	Total	**160**	**45**	**15**

S.XX. EXPERT REVIEW OF THE BENCHMARK AND EXTERNAL TEST-SET FREEZE

Protocol. An author who is a professional PLC programmer and industrial-automation engineer scored all 22 benchmark tasks against a ten-dimension rubric (Table S-XVII), assigning each dimension one of a three-point Acceptable/Minor-issue/Defect anchor grounded in control-engineering practice, plus a per-task overall verdict. This is an *author-side* review—the reviewer is an author of the paper—so it is a qualified-practitioner audit, not independent validation. The full package ships with the artifact (per-task review sheets, the score sheet, an independence protocol of solo scoring and lock-before-compare, and a `compute_agreement.py` tool reporting percent agreement and Cohen's/Fleiss' κ), so one or two external non-author engineers can later score the identical package and have their scores compared quantitatively; that independent multi-reviewer pass remains outstanding.

Aggregate outcome. Of 220 scored cells (22 tasks $\times$ 10 dimensions), 160 are Acceptable, 45 Minor issue, and 15 Defect (Table S-XVII). Overall verdicts are 1 Accept (`M04`), 18 Accept-with-fixes, and 3 Reject-level (`M06` burner purge, `H03` batch reactor, `H04` parking counter); 7 tasks carry a blocking defect (`E03`, `M02`, `M03`, `M06`, `M07`, `H03`, `H04`). Defect cells fall entirely on the substantive dimensions D3–D9—*zero* on D1 (requirement realism), D2 (polarity), and D10 (tier labels)—and property-completeness (D6) is the weakest dimension (5 Defect, 8 Minor). This separately corroborates the paper's own positive-obligation-scarcity / vacuity analysis (Section V-C, Sec. S.XV): the property suites, not the task premises, are where the benchmark is thinnest.

The seven blocking findings (one line each; full per-task errata in the artifact's `docs/17_BENCHMARK_ERRATA.md`):

- `E03` airlock: the spec text says door A wins a tie, but the reference keeps an already-open door B open (the safer behavior)—spec wording, not the code, is at fault.
- `M02` star–delta: the star→delta transfer happens in one scan with no dead-time property, so break-before-make is unverified (a real starter risks a phase short through the star arc).
- `M03` pedestrian crossing: no property forbids `CarYellow`∧`CarRed`, so code that lights both aspects passes all four properties (one-line fix: `G(!(CarYellow & CarRed))`).
- `M06` burner purge: a failed ignition returns to a reusable `Off` with no lockout state and no purge-before-fuel / flame-loss property or trace, so the central supervision behavior is never verified.
- `M07` two-hand control: the anti-tie-down property references only the internal `Armed` bit, so forcing `Armed` TRUE defeats the function yet still passes.
- `H03` batch reactor: the reaction dwell can complete with heater and agitator inhibited (contradicting the spec text), there is no over-temperature property, and no trace reaches `Drain`.
- `H04` parking counter: the `FullSign` property is one-way (a never-assert solution passes), there is no `Count` ≥ 0 lower bound, and simultaneous entry/exit edges at capacity net -1 (dropping `FullSign` while still full); the author note claiming otherwise is wrong.

Machine-green nuance, freeze, and errata release. All 22 references remain machine-green against the v1.0 property/scenario suites: every blocking finding is a place where the *formal suite under-constrains* the natural-language specification (or, for `E03` and the `H04` author note, where the spec wording itself is off), not a place where a reference violates its own encoded checks. Because the same v1.0 oracle scored every system, model comparisons are procedurally consistent (ranking invariance is not guaranteed, since models may differentially exploit under-constrained regions); only the absolute "verified-safe" reading requires the property-completeness qualification. The evaluated benchmark therefore stays *frozen at v1.0*, all findings are released as an errata document alongside the artifact, and nothing was retro-patched—silently patching the suites would invalidate every published number. A fix set may be applied only as a clearly versioned post-publication artifact update.

Masking sensitivity. Re-aggregating every metric over the 15 tasks that remain after excluding the seven review-flagged tasks (`E03`, `M02`, `M03`, `M06`, `M07`, `H03`, `H04`; 6 easy/7 medium/2 hard; `results/defect_mask_sensitivity.json`) preserves every substantive ranking. The API compile-and-invariant order is unchanged (grok-4.3† 0.760, claude 0.667, gpt-4o 0.147, gemini 0.127); among the trained conditions SFT-v2 stays strongest on all-checks-pass (0.464) and the gated-reward RL row highest on compile-and-invariant (0.504)—though the latter is effectively tied with SFT-v2 (0.502, Δ=0.002 at three seeds)—and the functional-reward gain over SFT-lite grows to +0.155 task-valid pass@1 (from +0.103 on all 22). The only ordering that flips is the near-tied gemini/gpt-4o bottom pair, which reorders across windows and task-sets—consistent with our declining to order it.

External test-set freeze. Separately, a 12-task family-disjoint external test set was expert-reviewed and frozen as v1.0 on 2026-07-25. Must-fix corrections were applied *before* freezing

TABLE S-XVIII
JUNE 2026 REPLICATION WINDOW: COMPILE-AND-INVARIANT PASS@1 BY TIER, WITH THE MATCHED SMV-ONLY COMPARISON. PRE-TRANSLATOR-REPAIR; PER-TASK ORACLE COUNTS RETAINED BUT NOT THE RAW PROGRAMS, HENCE NON-AUDITABLE. COLUMNS AS IN TABLE I OF THE MAIN PAPER; "RET." = DUAL / SMV-ONLY (CLAUDE 95%, GROK-4.3† 87%, GEMINI 61%, GPT-4O 31%—A ∼69% RELATIVE DROP). CONSISTENT WITH THE JULY PRIMARY PANEL AT THE AGGREGATE LEVEL (PER-TASK/TAXONOMY DETAIL IN THE ARTIFACT, `docs/07`). †`grok-4.3`, `reasoning_effort=none`, VIA THE RETIRED `grok-3` SLUG.

Model	Overall	Easy	Medium	Hard	SMV-only	Ret.
grok-4.3†	0.645	0.857	0.536	0.575	0.745	0.866
claude-sonnet-4-6	0.564	1.000	0.318	0.475	0.591	0.954
gemini-2.5-flash	0.095	0.200	0.000	0.175	0.155	0.613
gpt-4o	0.091	0.157	0.082	0.000	0.295	0.308

(the set was a draft and had never been evaluated): 3 reference-code defects were fixed (`X05`, a held `Reset` could defeat the jam latch; `X06` and `X11`, a trip could clear and restart in the same scan under a held start command), 12 properties were added to close verified holes, and 9 scenarios were added. The frozen set totals 12 tasks, 67 properties (from 55), 45 scenarios, and 13 positive-output obligations, and the harness re-verifies 12/12 references. A family-disjointness audit against all 22 benchmark tasks and all 11 SFT procedural families passed (one intra-set `X01`/`X08` staging-skeleton similarity is recorded as a diversity note), and a SHA-256 freeze manifest covers 25 files. This set is a held-out distribution-shift probe locked for a *single* confirmatory evaluation per system; it is deliberately *not* evaluated in this paper—no result on it exists—and any post-lock change would create a new version id. Deferring the single-shot run is what gives the set its confirmatory value: because it has never been scored, it has never influenced any development, architecture, or hyperparameter decision—preserving its confirmatory value—and the freeze protocol itself required professional review and sign-off, with all must-fix corrections above applied and the set re-verified before locking.

S.XXI. JUNE REPLICATION WINDOW

Table S-XVIII is the earlier June 2026 compile-and-invariant window, reported as a replication of the retained July primary panel (Table I of the main paper). It was measured *pre-translator-repair* (Sec. S.XVII) with the raw programs *not* retained, so it is non-auditable and historical only; the June per-task and failure-category detail for this window is released in the artifact (`docs/07`). At the aggregate level the two windows agree (claude compile-and-invariant 0.564 June vs. 0.555 July; grok-4.3† 0.645 vs. 0.677), which is what motivates treating July as primary while retaining June as a consistency check rather than a second independent estimate.